Guide to Using Python with Pandas

Table of Contents

Introduction

Pandas is a powerful data manipulation and analysis library for Python. It is widely used for data wrangling, cleaning, and analysis due to its intuitive data structures and easy-to-use functions.

Setting Up

Installing Pandas

Before using Pandas, you need to install it. You can install Pandas using pip:

```
pip install pandas
```

Importing Pandas

To use Pandas, you need to import it into your Python script or Jupyter Notebook:

```
import pandas as pd
```

Data Structures in Pandas

Pandas primarily uses two data structures:

1. **Series**: A one-dimensional labeled array capable of holding any data type.
2. **DataFrame**: A two-dimensional labeled data structure with columns of potentially different types.

Creating a Series

A Series can be created from a list, NumPy array, or a dictionary.

```
import pandas as pd

# Creating a Series from a list
data = [1, 2, 3, 4, 5]
series = pd.Series(data)
print(series)

# Creating a Series from a dictionary
data = {'a': 1, 'b': 2, 'c': 3}
series = pd.Series(data)
print(series)
```

Creating a DataFrame

A DataFrame can be created from a dictionary, list of dictionaries, or a NumPy array.

```
# Creating a DataFrame from a dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'San Francisco', 'Los Angeles']
}
df = pd.DataFrame(data)
print(df)

# Creating a DataFrame from a list of dictionaries
data = [
    {'Name': 'Alice', 'Age': 25, 'City': 'New York'},
    {'Name': 'Bob', 'Age': 30, 'City': 'San Francisco'},
    {'Name': 'Charlie', 'Age': 35, 'City': 'Los Angeles'}
]
```

```
df = pd.DataFrame(data)
print(df)
```

Basic Operations

Viewing Data

- **Head and Tail**: View the first or last few rows of the DataFrame.

```
print(df.head())  # Default is 5
print(df.tail(2))  # View last 2 rows
```

- **Info**: Get a summary of the DataFrame.

```
print(df.info())
```

- **Describe**: Get descriptive statistics for numerical columns.

```
print(df.describe())
```

Selecting Data

- **Selecting columns**:

```
print(df['Name'])  # Select single column
print(df[['Name', 'City']])  # Select multiple columns
```

- **Selecting rows by label**:

```
print(df.loc[0])  # Select first row by index
print(df.loc[0:1])  # Select first two rows by index
```

- **Selecting rows by position**:

```
print(df.iloc[0])  # Select first row by position
print(df.iloc[0:2])  # Select first two rows by position
```

Filtering Data

- **Filtering rows based on a condition**:

```
print(df[df['Age'] > 25])  # Select rows where Age > 25
```

Adding and Modifying Columns

- **Adding a new column**:

```
df['Country'] = ['USA', 'USA', 'USA']
print(df)
```

- **Modifying an existing column**:

```
df['Age'] = df['Age'] + 1
print(df)
```

Deleting Columns

- **Deleting a column**:

```
df = df.drop('Country', axis=1)
print(df)
```

Handling Missing Data

- **Identifying missing data**:

```
print(df.isnull())
print(df.isnull().sum())
```

- **Dropping missing values**:

```
df = df.dropna()
print(df)
```

- **Filling missing values**:

```

```

```
df = df.fillna(0)
print(df)
```

Advanced Operations

Grouping Data

Grouping data is useful for aggregating information based on certain criteria.

```
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Alice', 'Bob', 'Charlie'],
    'Year': [2020, 2020, 2020, 2021, 2021, 2021],
    'Sales': [250, 300, 400, 200, 350, 300]
}
df = pd.DataFrame(data)

grouped = df.groupby('Name').sum()
print(grouped)
```

Merging DataFrames

Merging allows you to combine two DataFrames based on a common column or index.

```
data1 = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
}
data2 = {
    'Name': ['Alice', 'Bob', 'David'],
    'Salary': [50000, 60000, 70000]
}

df1 = pd.DataFrame(data1)
df2 = pd.DataFrame(data2)

merged = pd.merge(df1, df2, on='Name', how='inner')
print(merged)
```

Saving and Loading Data

Reading Data from a File

Pandas can read data from various file formats including CSV, Excel, and SQL databases.

```

```

```
# Reading from a CSV file
df = pd.read_csv('data.csv')

# Reading from an Excel file
df = pd.read_excel('data.xlsx')

# Reading from a SQL database
import sqlite3
conn = sqlite3.connect('database.db')
df = pd.read_sql_query('SELECT * FROM table_name', conn)
```

Writing Data to a File

Pandas can write data to various file formats as well.

```
# Writing to a CSV file
df.to_csv('output.csv', index=False)

# Writing to an Excel file
df.to_excel('output.xlsx', index=False)

# Writing to a SQL database
df.to_sql('table_name', conn, if_exists='replace', index=False)
```

Conclusion

This guide provides an overview of using Pandas for data manipulation and analysis. By mastering these basic and advanced operations, you can efficiently handle and analyze large datasets in Python. Practice with different datasets and explore Pandas documentation for more functionalities and use cases. Happy coding!